

TANGO: Temporal-Spatial Co-Scheduling for Multi-SLO LLM Serving

Abstract—Online Large Language Model (LLM) serving faces the challenge of delivering high throughput while satisfying heterogeneous latency Service-Level Objectives (SLOs). Each request undergoes a prefill phase to produce the first token, followed by a decode phase that generates subsequent tokens, giving rise to two latency metrics: time-to-first-token (TTFT) and time-per-output-token (TPOT). However, existing LLM serving systems struggle to adapt resource allocation to the varying urgency of requests across prefill and decode phases under heterogeneous SLOs. We therefore propose TANGO, a temporal-spatial co-scheduling framework for Multi-SLO LLM serving. TANGO comprises a deadline-aware request dispatcher and a step-time-model-guided token allocator. The dispatcher prioritizes requests based on phase-aware SLO deadlines, while the token allocator regulates per-step workloads using predicted execution time. Together, they coordinate request ordering and batch composition to improve first-token responsiveness and inter-token timeliness. Experimental results show that TANGO improves overall SLO attainment by **58.72%**, and achieves **1.52×** higher goodput on average than state-of-the-art systems.

I. INTRODUCTION

Online large language model (LLM) services consolidate requests from multiple applications on shared model instances to improve GPU utilization and reduce serving costs. These applications impose distinct latency requirements: some requests are sensitive to the delay before the first output token, as in interactive chat, whereas others are sensitive to the delay between consecutive output tokens, as in real-time translation. These latency requirements are associated with two phases of LLM inference: during *prefill*, the model processes the prompt and generates the first output token; during autoregressive *decode*, it generates subsequent tokens one at a time. Time to first token (TTFT) measures the delay from request arrival until the first output token is returned, while time per output token (TPOT) measures the average delay per subsequent token. Online serving systems must therefore maintain high throughput while meeting application-specific TTFT and TPOT service-level objectives (SLOs).

Facing such a scenario, LLM serving aims to improve goodput by satisfying both TTFT and TPOT SLOs for requests with heterogeneous latency requirements. To support this goal, serving systems combine continuous batching, which dynamically admits new requests into the active batch between iterations, with chunked prefill, which divides long prompts into smaller segments that can be processed alongside decoding requests. This combination improves GPU utilization and serving efficiency and is adopted by state-of-the-art inference engines such as vLLM and SGLang.

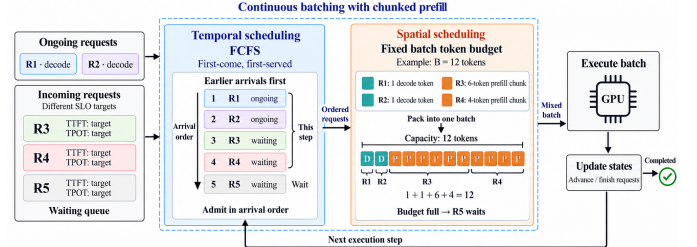


Fig. 1. Temporal and spatial scheduling under continuous batching and chunked prefill, illustrated with FCFS admission and a fixed token budget.

As shown in Fig. 1, incoming requests first enter the engine’s waiting queue. At each execution step, the scheduler selects waiting requests to run alongside ongoing requests. It then allocates the step’s token budget among these requests to form a batch, specifying the sizes of prefill chunks and the number of decode tokens to include. The engine then executes the batch, updates request states, and repeats this process for the next step. We abstract this process into two coupled scheduling dimensions: *temporal scheduling*, which determines request execution priority and admission, and *spatial scheduling*, which determines token allocation within each batch. These two decisions jointly shape request queueing delay and step duration, thereby affecting TTFT and TPOT.

Existing throughput-oriented engines commonly use first-come-first-served (FCFS) admission for temporal scheduling and a fixed maximum token budget for spatial scheduling, without adapting these policies to the heterogeneous SLOs of individual requests. To assess the impact of these policies, we evaluate the default FCFS scheduler on the Qwen3-8B model using a workload with heterogeneous TTFT and TPOT requirements. Among requests with tight TTFT SLOs, only **aa.bb%** satisfy both SLOs, with TTFT violations accounting for most failures. For requests with tight TPOT SLOs, joint SLO attainment is similarly low at **aa.bb%**, with TPOT violations as the main cause. Across the workload, only **aa.bb%** of requests satisfy both TTFT and TPOT SLOs. This low joint attainment highlights the limitations of SLO-unaware scheduling under heterogeneous latency requirements.

First, SLO-unaware temporal scheduling leads to an admission order that does not reflect request urgency. Under FCFS, a request with a tight TTFT objective can wait behind earlier requests with ample latency slack. The urgent request may exhaust its TTFT budget in the queue, even though the earlier requests could tolerate being served later. In our workload,

queueing accounts for **aa.bb%** of TTFT on average. FCFS therefore misses the opportunity to defer less urgent requests and use their slack to reduce queueing delays for requests with tighter SLOs.

Second, existing spatial scheduling policies allocate tokens under a fixed per-step budget without considering the TPOT requirements of active decode requests. Because decoding requests must wait for the entire batch to finish before receiving their next tokens, admitting more prefill work can prolong their inter-token delays. With 2,048 decode tokens fixed, increasing prefill work from zero to 2,048 tokens raises mean step time from **aa.bb ms** to **aa.bb ms**. Moreover, the same total token count can yield different step times: under a 4,096-token budget, pure-prefill and pure-decode steps both take approximately **aa.bb ms**, whereas an evenly mixed step takes **aa.bb ms**. A fixed token budget therefore cannot reliably bound step duration or adapt prefill allocation to the latency tolerance of decoding requests.

Addressing these inefficiencies requires coordinating request-level urgency with batch-level execution time. First, temporal scheduling involves deciding which waiting requests to admit and which ongoing requests can be deferred under resource contention. These requests are governed by different active SLOs and have consumed different portions of their latency budgets, so determining their execution order hinges on a consistent comparison of urgency across phases. Second, spatial scheduling couples the latency requirements of prefill and decode requests through token allocation within a shared batch. Allocating more tokens to prefill can advance requests with urgent TTFT objectives but also prolong the wait for decoding requests, whereas overly restricting prefill can leave those urgent requests waiting. Balancing these competing demands requires accurately estimating step time under varying prefill/decode compositions and execution contexts, then using that estimate to guide per-step token allocation.

We therefore propose TANGO, a multi-SLO LLM serving system that jointly controls temporal request dispatch and spatial token allocation. TANGO aims to improve goodput while satisfying heterogeneous TTFT and TPOT requirements. It comprises a deadline-aware request dispatcher and a step-time-model-guided token allocator. The deadline-aware request dispatcher assigns phase-aware deadlines according to each request’s active SLO and uses them to guide admission and preemption, prioritizing requests with tighter latency budgets. The step-time-model-guided token allocator uses a step-time model to predict the execution time of mixed prefill/decode batches and adjusts their prefill token budgets according to the remaining latency slack of decoding requests.

This paper makes three major contributions:

- **Investigating scheduling inefficiencies in multi-SLO LLM serving.** The analysis motivates us to account for heterogeneous request urgency and mixed prefill/decode interference through joint control of request dispatch and token allocation.
- **The design of a deadline-aware request dispatcher that prioritizes requests across inference phases.** By

TABLE I
COMPARISONS BETWEEN TANGO AND OTHER WORKS.

	Continuous Batching	Heterogeneous SLOs	Runtime Reprioritization	Predictive Token Budgeting
vLLM	✓			
SOLA	✓		✓	
JITServe	✓	✓	✓	
TANGO	✓	✓	✓	✓

mapping each request’s active SLO to a comparable deadline, it guides request admission and preemption, mitigating temporal inefficiency caused by SLO-unaware queueing.

- **The design of a step-time-model-guided token allocator that adapts batch composition to latency constraints.** Based on predicted step time, it converts the remaining TPOT slack into a prefill token budget, mitigating mixed prefill/decode interference while preserving progress for requests with tight TTFT requirements.

We evaluate TANGO against vLLM, SOLA, and JITServe using Llama-3.1-8B, Qwen3-14B, and Qwen3-32B under chatbot, mathematical-reasoning, and code-generation workloads. Compared with state-of-the-art systems, TANGO improves overall SLO attainment by **aa.bb%** and **aa.bb%**, and achieves up to **a.bb×** the goodput.

II. RELATED WORK

LLM Serving Systems. Recent studies have developed efficient LLM serving systems that improve inference throughput and latency through scheduling and memory optimizations. Orca introduced iteration-level scheduling, allowing requests to enter and leave a batch between decoding iterations to improve both utilization and responsiveness. vLLM further improved serving throughput through PagedAttention, which reduces KV-cache fragmentation and enables flexible memory sharing across requests. SGLang exploits prefix sharing and structured execution to accelerate multi-call LLM workloads. These systems establish continuous batching and efficient memory management as the foundation of modern LLM serving. However, throughput-oriented schedulers commonly use an SLO-agnostic request order and a fixed upper bound on the number of tokens processed in each iteration, without adapting either decision to the heterogeneous TTFT and TPOT requirements of individual requests.

Prefill–Decode Scheduling. The prefill and decode phases exhibit distinct execution characteristics: prefill processes many prompt tokens in parallel, whereas decode repeatedly processes one token per request. Mixing the two phases improves GPU utilization but can prolong an iteration and delay the next tokens of ongoing requests. Sarathi-Serve mitigates this interference through chunked prefill and stall-free batching, using decode-maximal batches to improve serving capacity under tail-latency constraints. DistServe instead places prefill and decode on different GPUs and separately optimizes their resource allocation and parallelism under TTFT and TPOT constraints. These approaches regulate interference through configured chunk sizes, phase-level batching policies,

or resource separation. They do not directly translate the remaining latency slack of active requests into an iteration-specific token budget for a colocated mixed batch.

SLO-Aware LLM Serving. Recent systems explicitly optimize the fraction of requests satisfying latency SLOs rather than raw throughput alone. SOLA formulates a fine-grained scheduling space that adjusts both request execution order and iteration workload according to request and system states, thereby balancing TTFT and TPOT. JITServe estimates uncertain request characteristics and allocates each request sufficient serving bandwidth while grouping compatible requests to improve batch efficiency and goodput. However, existing systems do not adjust the per-step token allocation within a mixed batch according to the request-specific TTFT and TPOT slack. Table I compares TANGO with representative serving systems.

III. BACKGROUND AND MOTIVATION

A. Multi-SLO LLM Serving

LLM Inference and Latency SLOs. An online LLM serving engine processes each request in two phases. During *prefill*, the engine processes all prompt tokens in parallel, constructs their key-value (KV) caches, and produces the first output token. During *decode*, it reuses the KV caches and autoregressively generates subsequent output tokens, typically one token per request in each iteration. The two phases expose different forms of latency and corresponding service-level objectives (SLOs): a request must first wait for its prompt to be processed, and must then make timely progress over a sequence of decoding iterations. Let request r_i arrive at time a_i and generate n_i output tokens, where $t_{i,j}$ denotes the time at which its j -th output token is returned. Its time to first token (TTFT) and time per output token (TPOT) are defined as

$$\begin{aligned} \text{TTFT}_i &= t_{i,1} - a_i, \\ \text{TPOT}_i &= \frac{1}{n_i - 1} \sum_{j=2}^{n_i} (t_{i,j} - t_{i,j-1}), \quad n_i > 1. \end{aligned} \quad (1)$$

TTFT captures the queuing and prefill delay before a response begins, whereas TPOT captures the average interval between consecutive tokens after generation starts. Each request is associated with a pair of latency objectives, denoted by S_i^{TTFT} and S_i^{TPOT} . Request r_i satisfies its SLOs only if $\text{TTFT}_i \leq S_i^{\text{TTFT}}$ and $\text{TPOT}_i \leq S_i^{\text{TPOT}}$. In this work, *Multi-SLO* refers both to these phase-specific objectives and to their heterogeneous values across requests and applications. Accordingly, serving efficiency is measured by goodput—the rate of requests that satisfy both objectives—rather than by raw request or token throughput alone.

Continuous Batching and Chunked Prefill. In static batching, a newly arrived request cannot join an executing batch until every request in that batch finishes, which wastes GPU capacity as requests complete at different times. Continuous batching instead updates the active batch at iteration boundaries. After each iteration, completed requests leave, unfinished decode requests remain active, and waiting requests

can be admitted into the released capacity. The batch composition can thus evolve throughout generation, improving GPU utilization while allowing the scheduler to reconsider which requests should run at every step.

A prefill request may contain hundreds or thousands of prompt tokens, whereas an active decode request contributes only its next token to an iteration. Scheduling an entire prompt at once can therefore produce a long iteration and delay token generation for the decode requests in the same batch. Chunked prefill divides the prompt into smaller chunks that may be processed over multiple iterations. This enables prefill chunks and decode tokens to be colocated in one batch, while exposing the prefill chunk size as a controllable scheduling variable. Specifically, let $\mathcal{P}_k$ and $\mathcal{D}_k$ denote the prefill and decode requests selected in step k , respectively, and let $c_{i,k}$ be the number of prompt tokens allocated to prefill request r_i . The number of tokens scheduled in this step is

$$T_k = \sum_{r_i \in \mathcal{P}_k} c_{i,k} + |\mathcal{D}_k|, \quad T_k \leq B_{\max}, \quad (2)$$

where each decode request contributes one token and $B_{\max}$ is the engine’s maximum per-step token budget. The resulting queuing delay and intervals between decoding iterations determine whether each request meets its TTFT and TPOT objectives.

B. Multi-SLO Traces for Investigations

We construct three representative LLM inference workloads covering conversational assistance, mathematical reasoning, and software-engineering agents.

Chatbots. We construct chatbot requests from LMSYS-Chat-1M, which contains one million real-world conversations collected from interactions with 25 LLMs. The token length of each recorded response determines its target output length.

Mathematical Reasoning. We use all 8,792 questions from the training and test splits of GSM8K. Each word problem serves as the input prompt, while the token length of its reference solution determines the target output length.

Software Agents. We use the software-engineering subset of AgentWorldBench and retain one request from each of 99 distinct trajectories. For each request, we generate 1,024 variants by prepending a distinct synthetic prefix while preserving the original agent action and target environment observation.

From the three request pools, we randomly sample 10,000 chatbot, 8,000 mathematical-reasoning, and 4,000 software-agent requests to form the replay traces.

Arrivals and SLOs. We associate requests in order with timestamps from failure-free entries in BurstGPT and replay each workload according to these timestamps, thereby preserving the trace’s temporal arrival pattern. For each workload, we first determine the sending rate λ_{90} at which vLLM achieves a 90% SLO attainment rate. We then set the sending rate to $1.2\lambda_{90}$ to emulate a workload under pressure.

For each request, we execute it alone on an otherwise idle vLLM deployment and record its measured TTFT and TPOT.

TABLE II
EXPERIMENT SPECIFICATIONS.

	Specifications
Hardware	2× Intel Xeon Gold 6530 CPUs, 64 physical cores 128 logical threads, 377.6 GiB host RAM (OS-reported) 4× RTX 6000 Ada GPUs (48 GB each); 2× GPUs used per run
Software	Qwen/Qwen3-14B (TP = 2); vLLM 0.17.0 PyTorch 2.10.0+cu128; CUDA 12.8; driver 570.124.06 LMSYS-Chat-1M, GSM8K, AgentWorldBench-SWE, BurstGPT

TABLE III
SLO ATTAINMENT OF vLLM WITH QWEN3-14B (TP = 2) ON THE MATH WORKLOAD AT $1.2\lambda_{90}$. JOINT ATTAINMENT REQUIRES SATISFYING BOTH THE TTFT AND TPOT OBJECTIVES.

SLO Class	TTFT (%)	TPOT (%)	Joint (%)
Overall	75.80	77.68	53.48
TTFT-prioritized	48.89	100.00	48.89
TPOT-prioritized	100.00	12.00	12.00
Balanced	57.39	100.00	57.39
Relaxed	100.00	100.00	100.00

These isolated measurements serve as request-specific latency baselines. We derive the two SLOs of request r_i as

$$\begin{aligned} S_i^{\text{TTFT}} &= c_i^{\text{TTFT}} \widehat{\text{TTFT}}_i, \\ S_i^{\text{TPOT}} &= c_i^{\text{TPOT}} \widehat{\text{TPOT}}_i, \end{aligned} \quad (3)$$

where $\widehat{\text{TTFT}}_i$ and $\widehat{\text{TPOT}}_i$ are the isolated-run measurements, and c_i^{TTFT} and c_i^{TPOT} are the coefficients assigned to the request’s SLO class. Using a fixed random seed, we assign requests equally among four SLO classes: TTFT-prioritized, TPOT-prioritized, balanced, and relaxed. For Qwen3-14B, the TTFT/TPOT coefficient pairs for these four classes are (20, 2.5), (50, 1.5), (25, 2), and (50, 3), respectively. All compared systems use the same request sequence, arrival schedule, and SLO targets under each experimental condition.

C. Low Multi-SLO Serving Performance

We use vLLM’s default FCFS scheduler as a representative serving baseline. It supports continuous batching and chunked prefill, but schedules requests without differentiating their TTFT and TPOT objectives. We run Qwen3-14B with TP = 2 on two GPUs using the configuration in Table II. We use the math workload as a representative case and replay it at its stress load of $1.2\lambda_{90}$.

Table III summarizes the SLO attainment rates for the math workload. Under FCFS, the overall TTFT and TPOT attainment rates are 75.80% and 77.68%, respectively, but only 53.48% of requests satisfy both objectives. The gap between individual and joint attainment shows that satisfying either latency objective alone does not provide effective Multi-SLO service.

The class-level results further reveal distinct failure modes. TTFT-prioritized requests achieve 48.89% TTFT attainment and 100.00% TPOT attainment, whereas TPOT-prioritized requests achieve 100.00% and 12.00%, respectively. Balanced and relaxed requests attain both objectives for 57.39% and

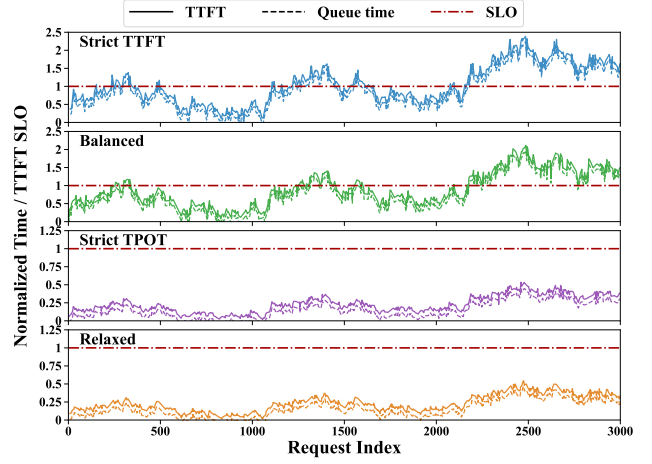


Fig. 2. TTFT and pre-admission queueing time for the math workload over request indices 0–3,000 (inclusive) under FCFS at $1.2\lambda_{90}$, normalized by each request’s TTFT SLO. Solid and dashed lines denote TTFT and queueing time, respectively; the dash-dotted line marks the SLO boundary. Strict TTFT and Strict TPOT correspond to the TTFT-prioritized and TPOT-prioritized classes, respectively.

100.00% of their requests, respectively. The contrasting class-level results indicate that the low joint attainment cannot be attributed to a single global latency bottleneck.

Together, these observations expose a mismatch between throughput-oriented scheduling and request-specific latency objectives. FCFS fixes the service order according to request arrival and does not account for the remaining TTFT or TPOT slack when admitting prefill chunks or advancing decode requests. Consequently, a scheduling decision that preserves prompt responsiveness may disrupt token delivery, while one that protects decode progress may increase the waiting time of queued prefills. The serving engine can therefore retain sufficient capacity to satisfy 100.00% of the relaxed requests, yet fail to direct that capacity toward the phase-specific objectives of other requests. These observations motivate a closer examination of how request ordering and per-step token allocation jointly affect Multi-SLO attainment.

D. Diving into Underlying Reasons

Our investigation identifies two complementary causes of the low Multi-SLO attainment: 1) FCFS distributes queueing delay without considering request-specific TTFT urgency, and 2) a fixed token budget does not provide a reliable latency bound for steps that mix prefill and decode tokens.

1) SLO-unaware temporal scheduling. The first-token latency of request r_i can be decomposed as

$$\text{TTFT}_i = W_i^{\text{queue}} + L_i^{\text{prefill}}, \quad (4)$$

where W_i^{queue} is the delay from arrival until the request is first admitted for prefill, and L_i^{prefill} is the remaining latency until its first token is produced. For a chunked prefill, L_i^{prefill} includes the execution and intervening scheduling delays of all prompt chunks after admission.

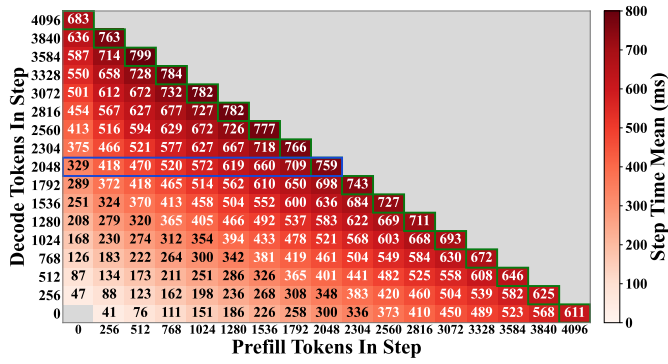


Fig. 3. Mean step time on Qwen3-14B with $TP = 2$, averaged over 50 timed step measurements per configuration (5 rounds $\times$ 10 measurements), under different numbers of prefill and decode tokens. The blue outline fixes 2,048 decode tokens; the green outlines mark configurations with the same 4,096-token total. Gray cells exceed the maximum per-step token budget.

Fig. 2 shows that the normalized queueing-time curve closely follows the TTFT curve in all four SLO classes, indicating that queueing dominates the first-token latency. For TTFT-prioritized and balanced requests, the queueing delay frequently consumes most, or even all, of the TTFT budget before prefill completes. TPOT-prioritized and relaxed requests remain below the boundary because their looser TTFT objectives provide substantially more slack. The resulting TTFT violations therefore arise primarily from how waiting time is distributed among requests, rather than from a uniform shortage of prefill compute capacity.

FCFS creates this imbalance because it admits requests according to arrival order, regardless of their remaining TTFT slack. A TTFT-prioritized request may wait behind earlier requests whose looser objectives could tolerate additional delay; the urgent request exhausts its latency budget in the queue while those ahead of it still finish within their SLOs. The temporal scheduling problem is thus to determine *which request* should receive prefill service first. Accounting for the remaining TTFT slack can redistribute queueing delay toward requests that can tolerate it, without changing the total amount of prefill work.

2) Workload-unaware spatial scheduling. The second failure mode arises after requests enter the running batch. For an active decode request, the interval before its next token is determined by the duration of the engine step in which that token is scheduled. Reordering waiting requests alone cannot protect TPOT because a decode token may still share a step with a large prefill chunk.

To quantify this interference, we vary the numbers of prefill tokens p and decode tokens d scheduled in the same step and measure the mean step time. As highlighted by the blue outline in Fig. 3, when $d = 2,048$, increasing p from 0 to 2,048 increases the mean step time from 329 ms to 759 ms. Every active decode request must wait for the longer step to complete before receiving its next token, directly increasing its TPOT.

More importantly, the total token count alone does not determine step time. Along the green boundary where $p + d =$

4,096, the pure-prefill and pure-decode configurations take 611 ms and 683 ms, respectively, while mixed configurations reach up to 799 ms. Thus, the same total token budget produces about a $1.3\times$ difference in latency. This nonlinearity reflects the distinct execution characteristics of compute-intensive prefill and memory-intensive decode, together with their interference when colocated in one iteration. Consequently, enforcing only $p + d \leq B_{\max}$ bounds the amount of scheduled work but cannot reliably bound the wall-clock latency experienced by active decode requests.

The spatial scheduling problem is therefore to determine *how many prefill tokens* can safely share the next step with the current decode workload. This requires an estimate of mixed-step latency and a mechanism that converts the remaining TPOT slack of active requests into a safe prefill allocation. Temporal request ordering and spatial token allocation address different failure modes: optimizing either dimension alone leaves the other source of SLO violations unresolved. These observations motivate their joint design in TANGO.

IV. TANGO METHODOLOGY

A. deadline-aware request dispatcher

B. step-time-model-guided token allocator

V. IMPLEMENTATION OF TANGO

VI. EVALUATION OF TANGO

VII. CONCLUSION